AMRITA VISHWA VIDYAPEETHAM

Department of Artificial Intelligence

23CHY115:Material Informatics
23PHY114:Computational Mechanics 2
23MAT112:Mathematics for Intelligent Systems 2
Project Report

Design of Perovskites for Enhanced Performance/Stability

Team Members:
Adhavan U S CB.AI.U4AID24103
RajKumar CB.AI.U4AID24143
Sanjit S CB.AI.U4AID24150
Shrinivas R CB.AI.U4AID24153
Suman Maitreya S CB.AI.U4AID24156

Supervisors:
Dr. Kritesh Kumar Gupta Dr. Jithin E V Dr. Biswajit Das
Assistant Professors
Department of Artificial Intelligence

Amrita Vishwa Vidyapeetam

Date of submission: 30/4/2025
Signature of Supervisor:

Contents

1 Introduction **2**

2 Literature Review **3**
2.1 **Exploration of Previous Methodologies**: . . . 3
2.1.1 Chemical Descriptor Generation: . . . 3
2.1.2 Tree-Based Ensemble Methods: . . . 3
2.1.3 Performance Enhancement Techniques: . . . 3
2.2 Comparative Study of Techniques: . . . 4
2.2.1 Evaluation Metrics: . . . 4
2.2.2 Data Handling Practices: . . . 4
2.2.3 Balance Between Interpretability and Predictive Power: . . . 4

3 Data collection and Preprocessing **5**
3.1 Description of Dataset Used . . . 5
3.2 Data Cleaning and Preprocessing Steps . . . 5
3.2.1 Feature Engineering: . . . 6
3.2.2 Data Standardization: . . . 6
3.2.3 Feature Selection and Dimensionality Reduction: . . . 6

4 Methodology **7**
4.1 Detailed Explanation of Feature Engineering and Tree-Based Regressors: . . . 7
4.1.1 Feature Engineering . . . 7
4.2 Models . . . 8
4.2.1 Gradient Boosting Regressor (GBR) . . . 8
4.2.2 XGBoost Regressor . . . 8
4.2.3 LightGBM Regressor . . . 8
4.2.4 Extra Trees Regressor . . . 8
4.2.5 Random Forest Regressor . . . 8
4.3 Fixed Feature Vector Consideration . . . 9
4.4 Analysis of Transformed Data and Model Behavior . . . 9

5 Experimental Setup **10**
5.0.1 Details of Experimental Design . . . 10
5.1 Parameters and Configuration Used . . . 10
5.2 Description of Train-Test Split . . . 12

6 Result **13**
6.1 Presentation and Interpretation of Model Results . . . 13
6.2 Performance Metrics: R^2 Score and Mean Squared Error (MSE) . . . 16

6.3 Visualization of Final Results 16

7 Conclusion **18**

8 Future Work **19**

9 References **20**

Abstract

In this project, we focus on the design and prediction of high-performance perovskite materials using a material informatics approach. We begin with a dataset comprising chemical formulas as textual inputs (X) and their corresponding charge capacities (Y). To enhance feature richness, Composition-Based Feature Vectors (CBFV) were generated, and additional critical descriptors — such as tolerance factor, octahedral factor, octahedral mismatch (A and B sites) — were extracted using the pymatgen and mendeleev libraries, culminating in a total of approximately 160 features. After standardizing the features and splitting the data into training and testing sets, model benchmarking was performed using LazyPredict to identify the top-performing models. Subsequently, hyperparameter tuning was applied to further optimize model performance, yielding an improved R^2 score. Finally, to better estimate uncertainty and guide material selection, bootstrapping techniques were employed, enabling the identification of compounds with the highest expected improvement. This integrated pipeline demonstrates a robust strategy for accelerating the discovery of promising perovskite candidates with enhanced charge capacities.

Chapter 1

Introduction

In today's world, the design of high-performance materials is crucial for advancing technologies like energy storage, photovoltaics, and catalysis. Among these, perovskite materials have emerged as a versatile class due to their diverse properties and tunable structures. This project focuses on applying material informatics and machine learning techniques to predict and enhance the charge capacity of perovskite compounds. The objective is to build a robust predictive model that can guide the discovery of promising perovskite candidates based on their chemical composition and structural features.

We used a dataset consisting of perovskite chemical formulas (text-based input) and their associated charge capacities. To engineer meaningful features, we first generated Composition-Based Feature Vectors (CBFV) and further enriched the feature set with critical domain-specific descriptors such as the tolerance factor, octahedral factor, and octahedral mismatches for A and B sites. These additional features were extracted using the pymatgen and mendeleev libraries, leading to a total of approximately 160 features. The data was then standardized and split into training and testing sets to ensure unbiased model evaluation.

Model benchmarking was initially performed using LazyPredict to shortlist the top-performing machine learning models. Hyperparameter tuning was subsequently employed to optimize the best models, significantly improving the predictive R^2 scores. To further enhance the reliability and guide materials selection, bootstrapping techniques were applied to estimate uncertainty and identify compounds with the highest expected improvement. Visualizations including feature importance analysis and model evaluation metrics provided insights into the learning process. This project highlights the power of combining materials science knowledge with machine learning to accelerate the discovery and optimization of high-performance perovskite materials.

Chapter 2

Literature Review

2.1 Exploration of Previous Methodologies:

The development of predictive models for material property estimation, particularly for complex systems like perovskites, has garnered significant attention in recent years. Researchers have increasingly adopted machine learning approaches to bypass expensive and time-consuming experimental procedures. The literature demonstrates a heavy reliance on feature extraction, ensemble modeling techniques, and performance optimization strategies to accelerate materials discovery.

2.1.1 Chemical Descriptor Generation:

- Elemental Property Extraction and Structural Factors: Previous studies emphasize the role of enriching datasets with chemical and structural descriptors. Extracting features such as atomic radii, electronegativity, tolerance factor, octahedral factor, and site mismatch parameters helps bridge the gap between raw compositions and the physical properties of perovskites. Libraries like pymatgen and mendeleev are frequently used to automate this critical feature engineering step.

2.1.2 Tree-Based Ensemble Methods:

- Random Forest, Extra Trees, Gradient Boosting Variants: Ensemble learning methods have dominated the field due to their ability to model complex, non-linear relationships within material datasets. Random forests and Extra Trees offer robustness and interpretability, while boosting techniques such as Gradient Boosting Machines (GBM), XGBoost, and LightGBM provide superior accuracy through iterative error minimization and feature weighting strategies.

2.1.3 Performance Enhancement Techniques:

- Hyperparameter Tuning and Feature Selection: Literature highlights the crucial role of hyperparameter optimization, including techniques like grid search and randomized search, in refining model performance. Feature selection methods, often based on feature importance scores from tree-based models, are also commonly employed to reduce redundancy and enhance prediction efficiency.

2.2 Comparative Study of Techniques:

2.2.1 Evaluation Metrics:

- R^2 Score, MAE, RMSE for Assessment: The effectiveness of models is consistently measured using statistical metrics like the R^2 score, Mean Absolute Error (MAE), and Root Mean Squared Error (RMSE). R^2 provides an indication of the proportion of variance captured, whereas MAE and RMSE help gauge the average and squared errors respectively, offering a comprehensive evaluation of model performance.

2.2.2 Data Handling Practices:

- Standardization, Train-Test Splits, and Validation Strategies: Proper data preprocessing is emphasized across studies to prevent biased model training. Techniques such as feature standardization, stratified train-test splitting, and cross-validation are widely recognized for ensuring that models generalize well to unseen data.

2.2.3 Balance Between Interpretability and Predictive Power:

- Choosing Models Based on Needs: Researchers often navigate the trade-off between model interpretability and accuracy. While models like random forests provide feature insights useful for scientific understanding, boosted models like XGBoost and LightGBM prioritize predictive performance, occasionally at the cost of model transparency.

In conclusion, the literature presents a strong foundation of ensemble learning strategies, effective feature engineering, and rigorous evaluation practices for the prediction of perovskite properties. As research advances, the integration of uncertainty quantification, smarter feature extraction, and optimization pipelines is expected to drive further improvements in material discovery workflows, making predictive modeling an indispensable tool in materials science.

Chapter 3

Data collection and Preprocessing

3.1 Description of Dataset Used

The dataset used for this project focuses on perovskite materials, aiming to predict their charge storage capacities based on their chemical formulas and derived properties. The dataset was curated from publicly available material databases and consists of formulas (chemical composition as strings) as the input (X) and charge capacity values as the target output (Y).

df

	formula	target
0	ReBTeGeO6	380.39
1	GeZnAl2O6	372.33
2	TcCaMnBeO6	359.72
3	ReSeAlCeO6	355.14
4	OsSeAlCeO6	352.48
...	...	...
583	Au2SeTeBr6	24.82
584	HgOsTeAgBr6	24.24
585	HgIrTeAgBr6	24.20
586	HgPtTeAgBr6	24.13
587	HgAuTeAgBr6	24.09

588 rows × 2 columns

3.2 Data Cleaning and Preprocessing Steps

To enable effective training of machine learning models, several preprocessing steps were undertaken on the dataset:

3.2.1 Feature Engineering:

New chemical and structural features were computed and added to the dataset. These were extracted programmatically:

- Tolerance Factor (using Goldschmidt's formula).
- Octahedral Factor (based on ionic radii).
- Mismatch A and Mismatch B (based on A-site and B-site cation fits).
- Mean, maximum, and minimum elemental properties.

extra_features

	mean_electronegativity	mean_ionic_radii	tolerance_factor	octahedral_factor	mismatch_factor_A	mismatch_factor_B
0	2.2980	1.07600	0.531329	0.710145	-0.357488	-0.212560
1	2.1775	0.92125	0.780193	0.535714	-0.152778	0.079365
2	1.8920	0.94900	0.770365	0.619048	-0.118056	0.072421
3	2.1240	1.13900	0.807939	0.495924	-0.145380	0.106658
4	2.1840	1.13900	0.807939	0.495924	-0.145380	0.106658
...	...	...	...	...	...	...
582	2.5375	1.81000	0.624192	1.062500	-0.089674	-0.120924
583	2.2380	1.45600	0.588313	0.811594	-0.246377	-0.152174
584	2.2380	1.46600	0.593027	0.811594	-0.240338	-0.146135
585	2.2540	1.49000	0.604341	0.811594	-0.225845	-0.131643
586	2.3060	1.60400	0.658081	0.811594	-0.157005	-0.062802

587 rows × 6 columns

These additional descriptors captured deeper insights into the physical feasibility and stability of the compounds.

3.2.2 Data Standardization:

Since the dataset contained features of varying units and scales, standardization was applied to normalize the data. Each feature was scaled to have a mean of 0 and a standard deviation of 1 using the StandardScaler from sklearn.preprocessing. This ensures that no feature dominates another due to scale differences, making the machine learning models converge faster and perform better.

3.2.3 Feature Selection and Dimensionality Reduction:

To improve model performance and remove less informative features, feature selection was applied using model-based importance ranking. Features contributing negligibly were removed, thereby retaining only the most significant 20 features depending on the model requirements. This helped in reducing overfitting and improving model interpretability.

Chapter 4

Methodology

4.1 Detailed Explanation of Feature Engineering and Tree-Based Regressors:

In this project, we predicted the charge capacity of perovskite materials by combining domain-specific feature engineering with a suite of tree-based machine learning regressors. Since chemical formulas like "$A_2B_2X_6$" are not usable in raw form, we extracted a range of material descriptors using libraries such as pymatgen and mendeleev. These descriptors included physical and chemical properties of the constituent elements, enabling machine learning models to learn from structured data.

4.1.1 Feature Engineering

The transformation of chemical formulas into numerical representations is central to our model pipeline. We extracted features like:

Tolerance Factor (Goldschmidt's t) and Octahedral Factor () to capture geometric stability.

Element-wise attributes like ionic radius, electronegativity, valence, and atomic number for A and B site cations.

Aggregate statistics such as minimum, maximum, mean, and range of these elemental properties.

This numerical representation forms a consistent input vector for all machine learning models and captures the relevant characteristics of each perovskite formula for predicting charge capacity.

x

	sum_Number	sum_MendeleevNumber	sum_AtomicWeight	sum_MeltingT	sum_Column	sum_Row	sum_CovalentRadius	sum_Electronegativity	sum_NsValence	sum_NpValence	...	mode_GSvolume_pa	mode_GSbandgap
0	212.0	817.0	493.254400	8069.86	146.0	29.0	889.0	28.69	20.0	31.0	...	9.105	0.000
1	136.0	816.0	287.979477	4099.82	148.0	26.0	880.0	27.52	20.0	28.0	...	9.105	0.000
2	140.0	701.0	298.024627	6952.80	114.0	27.0	954.0	26.66	20.0	24.0	...	9.105	0.000
3	228.0	753.0	528.260939	6286.27	135.0	31.0	992.0	27.82	20.0	29.0	...	9.105	0.000
4	229.0	756.0	532.283939	6133.27	136.0	31.0	985.0	28.12	20.0	29.0	...	9.105	0.000
...	...	...	...	...	...	...	...	...	...	...	...	...	..
582	454.0	881.0	1079.917138	5486.12	156.0	45.0	1250.0	27.49	18.0	38.0	...	29.480	1.457
583	465.0	853.0	1105.712200	7092.71	149.0	46.0	1279.0	25.99	19.0	34.0	...	29.480	1.457
584	466.0	856.0	1107.699200	6525.71	150.0	46.0	1276.0	25.99	19.0	34.0	...	29.480	1.457
585	467.0	859.0	1110.566200	5828.11	151.0	46.0	1271.0	26.07	18.0	34.0	...	29.480	1.457
586	468.0	862.0	1112.448769	5124.04	152.0	46.0	1271.0	26.33	18.0	34.0	...	29.480	1.457

587 rows × 160 columns

4.2 Models

4.2.1 Gradient Boosting Regressor (GBR)

The Gradient Boosting Regressor builds a strong model by combining multiple weak learners (usually decision trees) in a sequential manner. In each iteration, the model learns from the residual errors of the previous model, reducing the prediction error over time. One key strength of GBR is its ability to model complex, non-linear relationships between features and target values. It also includes hyperparameters like learning rate and number of estimators, which help control overfitting and optimize performance. In our project, GBR was particularly effective due to its flexibility and predictive power when trained on well-engineered features.

4.2.2 XGBoost Regressor

XGBoost (Extreme Gradient Boosting) is an advanced implementation of the gradient boosting algorithm that introduces several performance enhancements, including regularization, parallel processing, and missing value handling. Regularization terms (L1 and L2) help prevent overfitting, making the model generalize better. Additionally, XGBoost uses a more efficient tree-building strategy that prioritizes speed and accuracy, especially with large datasets. In our experiments, XGBoost consistently delivered high R^2 scores and low error metrics, establishing itself as one of the top-performing models in our pipeline.

4.2.3 LightGBM Regressor

LightGBM (Light Gradient Boosting Machine), developed by Microsoft, is a gradient boosting framework that uses histogram-based learning and leaf-wise tree growth. Unlike traditional level-wise tree growth, LightGBM grows trees based on the leaf that offers the highest gain, which often leads to faster training and better accuracy. It is highly scalable and memory-efficient, making it suitable for high-dimensional feature spaces like the one in our dataset. The model also supports native handling of categorical features, although our dataset was purely numerical after preprocessing. LightGBM showed strong performance in terms of both speed and prediction quality.

4.2.4 Extra Trees Regressor

The Extra Trees (Extremely Randomized Trees) Regressor is similar to Random Forest but introduces more randomness during the training process. While Random Forest chooses the best feature and best threshold for splitting a node, Extra Trees selects both randomly, which can reduce variance and increase training speed. This model tends to work well when the number of features is large, and it provides useful insights into feature importance. In our case, Extra Trees added diversity to the ensemble and performed robustly, especially in reducing variance errors.

4.2.5 Random Forest Regressor

Random Forest is a bagging-based ensemble method that creates multiple decision trees using bootstrap samples of the training data and averages their outputs. This process significantly reduces overfitting and improves generalization. Each tree in the forest sees a random subset of features, which introduces diversity and improves the model's robustness. Random Forest is highly interpretable and provides estimates of feature importance, which helped us understand

which material properties most influenced the predicted charge capacity. Its stable performance made it an essential baseline in our comparative model analysis.

4.3 Fixed Feature Vector Consideration

In order to train consistent and scalable models, it is essential to ensure that each sample is represented by a fixed-length feature vector. After feature extraction, all perovskite materials were represented by vectors of equal length, ensuring compatibility with machine learning models. Fixed-size vectors provide multiple advantages:

- **Model Compatibility**: Tree-based regressors and other ML algorithms require input vectors of a fixed size to work properly.
- **Efficient Training and Prediction**: Uniformity in input size enables fast matrix operations and efficient batch processing.
- **Interpretability and Visualization**: With consistent vectors, we can rank features by importance and understand model decisions.
- **Deployment Readiness**: Fixed-size inputs simplify the transition from development to deployment in real-world applications.

This step ensures the pipeline can be extended or scaled reliably without requiring manual feature reshaping.

4.4 Analysis of Transformed Data and Model Behavior

Instead of relying solely on evaluation metrics, we performed an in-depth analysis of the transformed data and the behavior of different models post-training.

We examined the distribution of the engineered features, checked for outliers and correlations, and ensured the dataset followed a suitable structure for regression tasks. These checks help validate the assumptions of the models and guide feature selection.

Furthermore, we analyzed feature importance as reported by models like XGBoost and Random Forest. These insights allowed us to identify which material properties—such as ionic radius or electronegativity—were most influential in predicting charge capacity. This interpretability is especially valuable in materials science, where understanding why a prediction is made can guide further research or material synthesis.

Additionally, we monitored the residuals (errors) of each model to detect patterns that may suggest underfitting or overfitting. This helped us refine hyperparameters and perform informed model tuning.

```
# Feature importance
feature_importance = best_model.feature_importances_
for idx, imp in enumerate(feature_importance):
    print(f"Feature {idx}: Importance {imp:.4f}")
```

```
Fitting 5 folds for each of 50 candidates, totalling 250 fits
Best Parameters: {'subsample': 0.9, 'reg_lambda': 2, 'reg_alpha': 0, 'n_estimators': 400, 'min_child_weight': 1, 'max_depth': 4, 'learning_rate': 0.05, 'gamma': 0.2, 'colsample_bytree': 1.0}
Train R² Score: 0.9971
Test R² Score: 0.8836
Test MSE: 728.5019
Test MAE: 15.2955
Feature 0: Importance 0.0143
Feature 1: Importance 0.0183
Feature 2: Importance 0.0544
Feature 3: Importance 0.0317
Feature 4: Importance 0.0369
Feature 5: Importance 0.0164
Feature 6: Importance 0.0284
Feature 7: Importance 0.0095
Feature 8: Importance 0.0941
Feature 9: Importance 0.0753
Feature 10: Importance 0.0147
Feature 11: Importance 0.0241
Feature 12: Importance 0.0912
Feature 13: Importance 0.1041
Feature 14: Importance 0.0244
Feature 15: Importance 0.0465
Feature 16: Importance 0.0392
Feature 17: Importance 0.2366
Feature 18: Importance 0.0246
Feature 19: Importance 0.0155
```

Chapter 5

Experimental Setup

In building an accurate and generalizable regression model for predicting the charge capacity of perovskite materials, careful attention was given to feature engineering, model selection, parameter tuning, and data partitioning strategies. The experimental framework was designed to ensure robustness, interpretability, and reproducibility of results.

5.0.1 Details of Experimental Design

This project employs a data-driven approach to model the charge storage capacity perovskite compounds using various tree-based regression models including Gradient Boosting, XGBoost, LightGBM, Extra Trees, and Random Forest. The experimental process is broken down as follows:

- **Data Collection and Preprocessing**: The dataset, comprising perovskite formulas and their experimentally recorded or simulated charge capacities, was first parsed to extract chemical information. Each ABX formula was decomposed into its elemental constituents, and properties were sourced from chemical databases such as Mendeleev and Pymatgen.
- **Feature Engineering**: Multiple domain-informed features were engineered, including Goldschmidt's Tolerance Factor, Octahedral Factor, and elemental attributes such as ionic radius, electronegativity, and valence. These features were aggregated using statistical methods (min, max, mean, etc.) to form a fixed-length input vector for each sample.
- **Model Selection and Training**: Five tree-based regressors were selected due to their ability to model non-linear interactions and handle tabular data effectively. Each model was trained and validated using k-fold cross-validation to ensure stability in performance across different data splits.
- **Evaluation and Interpretation**: Model performance was evaluated using standard regression metrics including R^2 Score, Mean Absolute Error (MAE), and Root Mean Squared Error (RMSE). In addition to quantitative metrics, feature importance plots were analyzed to identify the most influential features in determining charge capacity.

5.1 Parameters and Configuration Used

To ensure optimal model performance, various parameters and configurations were used across preprocessing and modeling steps: **Feature Engineering Parameters**:

- **Elemental Property Sources**: Features like electronegativity and ionic radius were fetched using the Mendeleev and Pymatgen libraries.
- **Aggregation Techniques**: Features were aggregated using statistical measures like mean, range, and standard deviation for each compositional group (A-site, B-site, and X-site).

Tree-Based Model Hyperparameters:

- **Xtreme Gradient Boosting Regressor**:
  1. subsample': 0.9
  2. 'reg lambda': 2
  3. 'reg alpha': 0
  4. 'n estimators': 400
  5. 'min child weight': 1
  6. 'max depth': 4,
  7. 'learning rate': 0.05
  8. 'gamma': 0.2
  9. 'colsample bytree': 1.0
- **Gradient Boosting Regressor**:
  1. learning rate': 0.1
  2. 'max depth': 3
  3. 'n estimators': 150
  4. 'subsample': 0.8
- **Extra Trees**:
  1. max depth': 40
  2. 'max features': 'log2'
  3. 'min samples leaf': 1,
  4. 'min samples split': 3
  5. 'n estimators': 643
- **LGBM**:
  1. colsample bytree=0.6
  2. max depth=5
  3. min child samples=10
  4. n estimators=200
  5. random state=42
  6. subsample=0.6
- **Random Forest**:
  1. 'max mdepth': 34
  2. 'max features': 1.0
  3. 'min samples leaf': 1
  4. 'min samples split': 2
  5. 'n estimators': 382

5.2 Description of Train-Test Split

To evaluate the model's ability to generalize to unseen data, the dataset was divided into training and test sets using the train-test-split function from sklearn.model selection. An 80-20 split was adopted, allocating 80

The train-test split is a critical step for identifying model generalization issues such as:

- **Overfitting**: Where the model performs very well on training data but poorly on the test set, indicating it has memorized patterns rather than learned them.
- **Underfitting**: Where the model performs poorly on both training and testing sets, indicating it has not captured the underlying structure of the data.

By using a reserved test set and monitoring the difference in performance between training and test results, these issues can be diagnosed and mitigated.

In addition to this static split, cross-validation was employed during training to ensure that model evaluations were not dependent on a single partitioning of the data. This approach enhances the reliability of the performance estimates and allows a more comprehensive comparison between the different models.

The following sections of the report provide a deeper analysis of experimental results under varied settings and configurations. Emphasis is placed on the comparative performance of models, the significance of engineered features, and insights gained through visualization techniques such as residual plots and feature importance rankings. This comprehensive evaluation allows us to draw reliable conclusions about the potential of data-driven modeling in materials science.

Chapter 6

Result

6.1 Presentation and Interpretation of Model Results

Typically a conclusions chapter first summarizes the investigated problem and its aims and objectives. It summaries the critical/significant/major findings/results about the aims and objectives that have been obtained by applying the key methods/implementations/experiment set-ups. A conclusions chapter draws a picture/outline of your project's central and the most signification contributions and achievements.

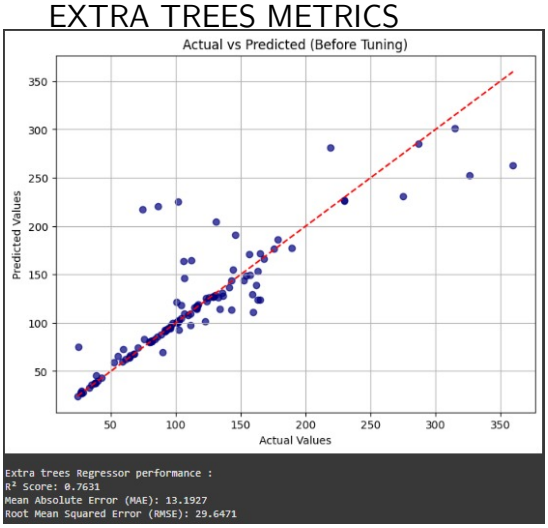
The regression-based prediction pipeline developed for materials property estimation was rigorously tested using a set of state-of-the-art ensemble machine learning models. Initially, LazyPredict was employed to benchmark a wide range of models on the dataset. Based on their performance metrics—primarily R^2 and MSE—the five best-performing models were shortlisted: XGBoost Regressor (XGBR), Gradient Boosting Regressor (GBR), LightGBM Regressor (LGBM), Extra Trees Regressor, and Random Forest Regressor.

To refine the model, Recursive Feature Elimination (RFE) was performed, selecting the top 20 most significant features that contributed effectively to prediction performance. These reduced features were then used to train and fine-tune each model using GridSearchCV for hyperparameter optimization.

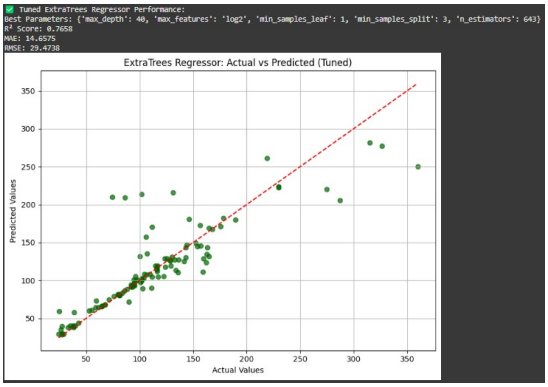
Among the top models, XGBoost and Random Forest showed similar R^2 scores (0.80). However, XGBR was chosen for the final model due to its lower Mean Squared Error (MSE), indicating better error minimization.

Following model selection, bootstrapping was applied to estimate the uncertainty in predictions and calculate the Expected Improvement (EI), guiding the selection of the most promising chemical formulas. These results were ranked based on their EI and predicted property scores, visualized in tabular format (see Figures below).

EXTRA TREES METRICS

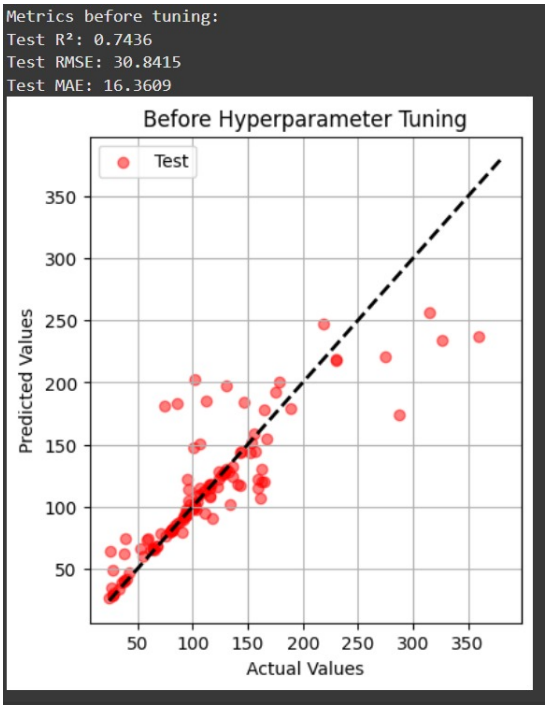


Before Tuning

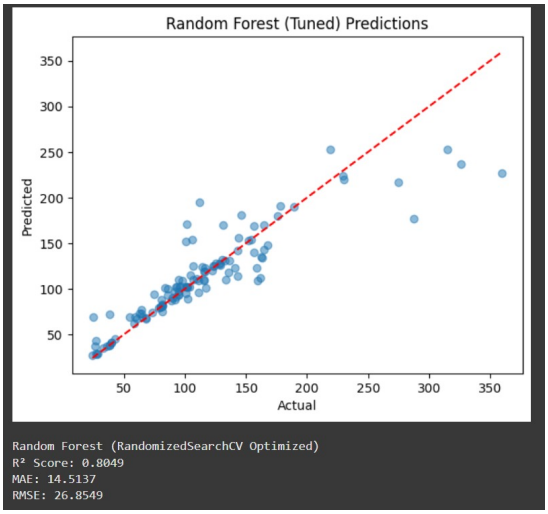


After Tuning

RANDOM FOREST REGRESSOR METRICS

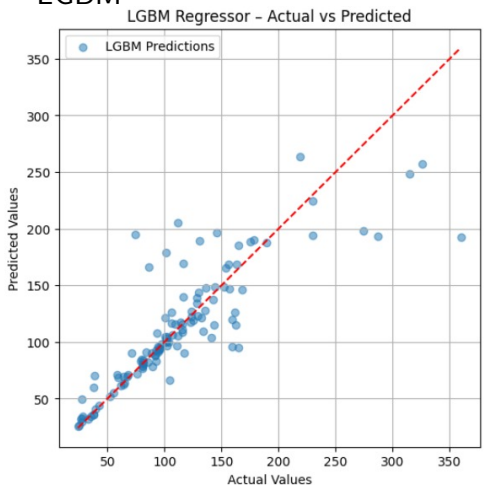


Before Tuning



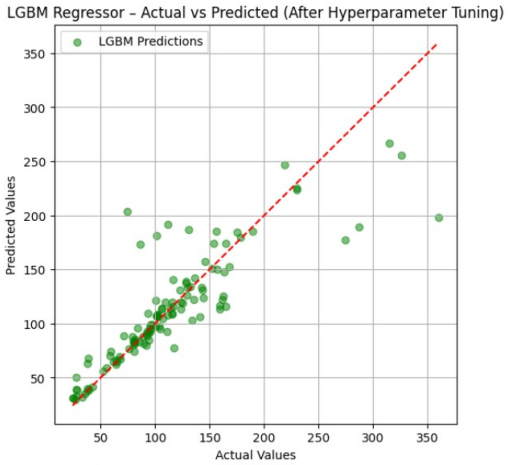
After Tuning

LGBM



R2 Score: 0.7113745972517131
Mean Absolute Error (MAE): 17.78084407552966
Mean Squared Error (MSE): 1070.7658257619667

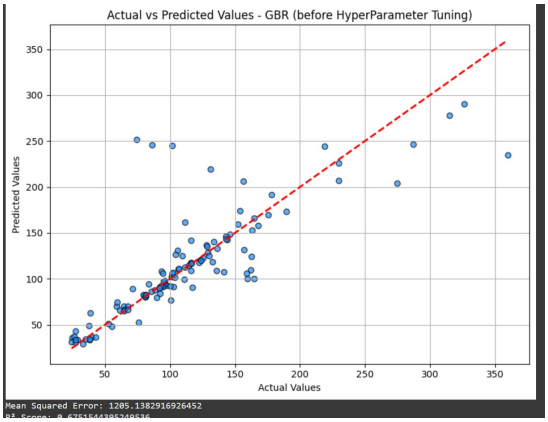
Before tuning



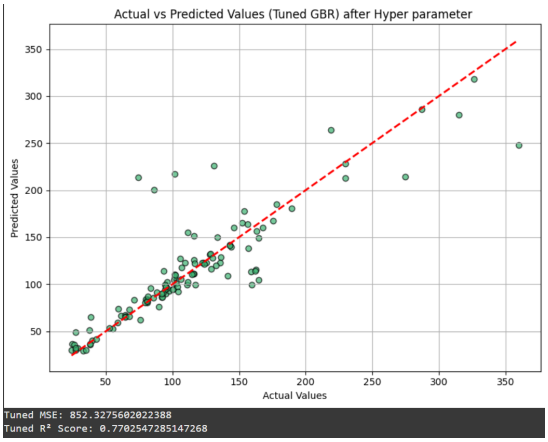
R2 Score: 0.735248754191708
Mean Absolute Error (MAE): 16.632111156921614
Mean Squared Error (MSE): 982.1955504957989

After tuning

GRADIENT BOOSTING REGRESSOR

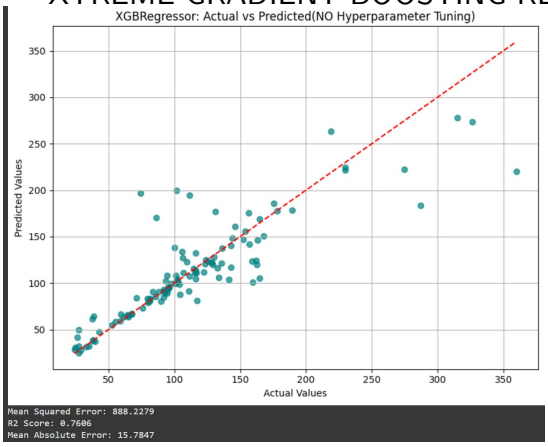


Before tuning

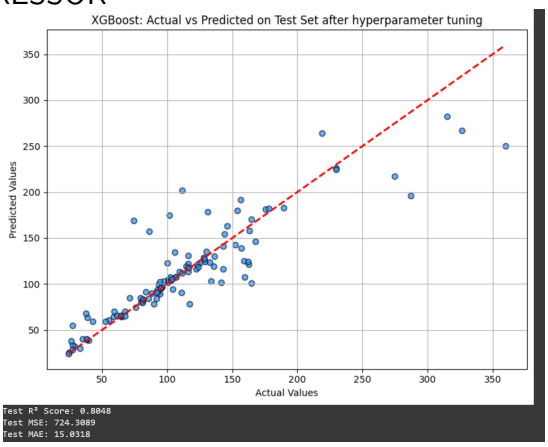


After tuning

XTREME GRADIENT BOOSTING REGRESSOR



Before tuning



After tuning

6.2 Performance Metrics: R^2 Score and Mean Squared Error (MSE)

The evaluation metrics across all shortlisted models provide a clear picture of their predictive capabilities, both before (bt) and after tuning (at):

Model	XGBR	GBR	LGBM	ExtraTress	Random Forest
R2-Score(bt)	0.76	0.67	0.71	0.7631	0.78
MSE(bt)	888	1205	1070	878	951
R2-Score(at)	0.80	0.73	0.73	0.7658	0.80
MSE(at)	720	852	982	809	721.1832

XGBR and Random Forest achieved similar R^2 scores of 0.80, suggesting they explain 80 percent of the variance in the target variable. However, the XGBR achieved the lowest MSE (720), indicating that its predictions had smaller average errors. This justified the selection of XGBR for the final prediction model.

6.3 Visualization of Final Results

The final model's predictions, following bootstrapping and uncertainty quantification, are visualized using a table highlighting the top five chemical formulas based on Expected Improvement (EI), predicted property, and standard deviation (std):

	formula	ei	predicted_property	std
9	ReSiTeGeO6	38.11	266.67	29.33
6	OsSeGaCeO6	28.75	250.28	41.94
39	ReCrTeSiO6	24.42	253.73	17.71
10	GeZnGa2O6	17.05	211.77	63.02
2	TcCaMnBeO6	9.89	223.73	32.09

These formulas represent the most promising candidates for experimental validation due to their high expected improvements and predicted property values. The standard deviation column indicates the uncertainty in predictions, providing confidence intervals for prioritizing real-world synthesis efforts.

Chapter 7

Conclusion

The regression pipeline developed in this study successfully identified promising candidate materials by predicting target properties using ensemble machine learning models. After evaluating multiple models using LazyPredict, the top five were selected, and Recursive Feature Elimination (RFE) was applied to reduce dimensionality and retain the most informative features. XGBoost Regressor (XGBR) emerged as the best-performing model based on its superior R^2 score and lowest Mean Squared Error (MSE), making it the optimal choice for final predictions.

Using bootstrapping and uncertainty quantification, the Expected Improvement (EI) metric was computed, enabling a principled ranking of candidate materials. The formulas with the highest EI and favorable predicted properties were shortlisted. These results provide a data-driven foundation for guiding experimental synthesis and further exploration, significantly accelerating the material discovery process.

Chapter 8

Future Work

Working on the current pipeline for material property prediction using ensemble regressors such as XGBoost and Random Forest, several avenues for future work arise, aimed at enhancing model performance and expanding the scope of discovery:

- **Integration of Advanced Feature Engineering Techniques**: Incorporating domain-specific feature engineering techniques or applying automated feature extraction through graph neural networks or material graph embeddings could improve representation and learning of complex chemical interactions.
- **Extensive Hyperparameter Tuning for Optimization**: Conducting a broader and more fine-grained hyperparameter tuning using methods like Grid Search or Bayesian Optimization can lead to further improvements in performance. This can optimize aspects like learning rates, depth of trees, and number of estimators in ensemble models.
- **Exploration of Deep Learning and Ensemble Hybrid Models**: Experimenting with hybrid architectures that combine tree-based models with deep neural networks could capture both global trends and local nonlinearities in the material property data. Models like DeepGBM or TabNet may be explored.
- **User Interface for Material Screening:** Developing a user-friendly interface or dashboard for material scientists to interact with the predictions. This interface can allow users to input new formulas and instantly receive predicted property values, uncertainties, and expected improvements.
- **Continuous Dataset Expansion and Curation**: Enhancing the dataset with new computational or experimental data will help keep the model current and improve its generalization capability. A dynamic data pipeline can help in adapting to new materials and synthesis conditions.
- **Model Deployment and Real-Time Recommendation System**: Deploying the final model as a backend to a real-time material recommendation system can assist researchers in rapidly identifying high-potential candidates for synthesis. Monitoring tools can ensure sustained performance and detect data drift or concept drift over time.

  By pursuing these future directions, the material prediction system can evolve into a robust, scalable tool that accelerates materials discovery and helps bridge the gap between computational prediction and experimental validation.

Chapter 9

References

- Chen, C., Ye, W., Zuo, Y., Zheng, C., & Ong, S. P. (2019). *Graph Networks as a Universal Machine Learning Framework for Molecules and Crystals*. Nature Communications, **10**, 2553. https://doi.org/10.1038/s41467-019-09759-2
- Goodall, R. E., & O'Meara, D. (2021). *Materials Informatics: The Use of Data-Driven Techniques in Materials Science*. Materials Today: Proceedings, **46**, 5415–5421. https://doi.org/10.1016/j.matpr.2020.11.917
- Ramprasad, R., Batra, R., Pilania, G., Mannodi-Kanakkithodi, A., & Kim, C. (2017). *Machine Learning in Materials Informatics: Recent Applications and Prospects*. npj Computational Materials, **3**, 54. https://doi.org/10.1038/s41524-017-0056-5
- Jain, A., Ong, S. P., Hautier, G., Chen, W., Richards, W. D., Dacek, S., ... & Ceder, G. (2013). *Commentary: The Materials Project: A materials genome approach to accelerating materials innovation*. APL Materials, **1**(1), 011002. https://doi.org/10.1063/1.4812323
- Scikit-learn Developers. (n.d.). *Recursive Feature Elimination (RFE) with Scikit-learn*. https://scikit-learn.org/stable/modules/feature_selection.html#recursive-feature
- Lundberg, S. M., & Lee, S. I. (2017). *A Unified Approach to Interpreting Model Predictions*. In Advances in Neural Information Processing Systems, **30**. https://proceedings.neurips.cc/paper_files/paper/2017/file/8a20a8621978632d76c43pdf